



EDA Ene2016

Estructuras de Datos Avanzadas (Universidad Rey Juan Carlos)



Scan to open on Studocu

Estructuras de Datos Avanzadas

Examen Ordinario

11 de enero de 2016

Para la realización de los siguientes ejercicios se suministra un proyecto que contiene la cabecera de todas los métodos que se solicitan así como diferentes test unitarios que pueden ayudar en el desarrollo.

El tiempo para el desarrollo de esta prueba es de 1 hora y 30 minutos.

Ejercicio 1 [3 puntos]

Se desea aumentar la funcionalidad de `LinkedBinaryTree` incorporando la siguiente funcionalidad:

- a) **[1 punto]** Se dice que un árbol binario es perfecto si todos los nodos internos tienen dos hijos. Implementar un método `isPerfect`, que determine si un árbol binario es perfecto o no.
- b) **[1 punto]** Se dice que un árbol binario es zurdo si es vacío, solo tiene el nodo raíz o más de la mitad de sus descendientes están en el hijo izquierdo. Implementar el método `isOdd`, que determina si un árbol binario es zurdo o no.
- c) **[1 punto]** `InternalNodeIterator`: clase que permite recorrer los nodos internos de un árbol binario.

NOTA: Se valorará un buen diseño (de clases y métodos) para obtener la máxima calificación.

Las cabeceras de los métodos solicitados están al final del fichero "`LinkedBinaryTree.java`". Y la clase `InternalNodeIterator` está en el mismo paquete.

Ejercicio 2 [4 puntos]

La empresa *URJC Investment* ha adquirido recientemente la Torre Europa. Su intención es alquilar las oficinas de dicha torre a distintas empresas. Para facilitar el trabajo del personal de recepción, *URJC Investment* desea un módulo en Java que permita buscar con facilidad información de los trabajadores y de las empresas que tengan sus oficinas en dicha torre. Para ello, se creará una clase *URJCInvest* que permitirá localizar, por el nombre, cualquiera de las empresas presentes en el edificio, y cualquier trabajador que trabaje en una de esas empresas.

Cada trabajador se modelará con una clase *Employee* que debe contener la siguiente información: nombre de la empresa, nombre del empleado y cargo que ocupa en la empresa.

Para cada empresa, la clase *URJCInvest* almacenará un organigrama con la relación que existe entre los trabajadores. Cada organigrama se modelará mediante una clase *OrganizationChart*, que contendrá la estructura de datos necesaria.

Por simplicidad, se supondrá que cada empresa tiene un único CEO, que es jefe de los directivos de máximo nivel. Además, cada trabajador siempre dependerá de un único jefe. Por último, se considera que todos los trabajadores tienen nombres diferentes, y que un trabajador no puede estar en más de una empresa.

Se pide:

- a) **[1.5 punto]** Implementar el método `insertEmployee` que inserta un empleado en *URJCInvest*, dicho método recibe cadenas con los datos del empleado y el nombre del jefe directo. Además, definir los atributos necesarios de las clases *URJCInvest* y *Employee* para almacenar la información requerida de tal forma que se minimice la complejidad algorítmica de la operación solicitada.
- b) **[1 punto]** Implementar en *URJCInvest* el método `searchCompany`, que reciba el nombre de una empresa y devuelva un objeto de tipo *OrganizationChart* con el organigrama correspondiente, o `null` si la empresa no existe. Obsérvese que, para almacenar la información requerida de tal forma que se minimice la complejidad algorítmica de la operación solicitada, quizás tenga que añadir código al método `insertEmployee` y definir nuevos atributos en *URJCInvest* y en *OrganizationChart*.
- c) **[1.5 punto]** Implementar en *OrganizationChart* el método `getGrantHolders` que devuelva la colección de personal de mínima responsabilidad (que no es jefe de nadie) de dicho organigrama. Se valorará la elección de la colección de retorno adecuada para permitir accesos aleatorios.

NOTA: En la clase *URJCInvest* y en *OrganizationChart* se podrán agregar los métodos que se consideren oportunos y cualquiera de las estructuras de datos estudiadas durante el curso, valorándose especialmente lo adecuado de la elección que se realice. Sin embargo, se prohíbe modificar de cualquier forma las cabeceras de los métodos suministrados en el paquete *urjcinvestment* incluido en el proyecto.